CHOGOU Donald

```
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
```

1. Histogramme d'amplitude et histogramme cumulé

Cellule 2 : Fonction pour vérifier que l'image est bien en niveau de gris

```
def est_niveau_de_gris(img):
    """
    Vérifie rigoureusement si une entrée (objet PIL ou matrice NumPy)
    est une image en niveaux de gris.
    """
    if isinstance(img, Image.Image):
        return img.mode in ['L', 'LA', 'I', 'F']
    elif isinstance(img, np.ndarray):
        return img.ndim == 2 or (img.ndim == 3 and img.shape[2] == 1)
```

Cellule 3 : Algorithme de calcul des deux histogrammes

Conformément aux consignes, cette fonction prend en entrée la matrice de gris I et le niveau de décomposition NIV. Si NIV est inférieur à 256, les intensités sont automatiquement quantifiées pour s'adapter au nombre de boîtes demandé.

```
def compute_histograms(I, NIV):
    # 1. Aplatir la matrice ET convertir en entiers standards (int) pour éviter l'overflow
    img_flat = I.flatten().astype(int)

    # 2. Initialiser le tableau de l'histogramme selon la dimension NIV demandée
    hist = np.zeros(NIV)

    # 3. Parcourir chaque pixel et incrémenter la bonne boîte de décomposition
    for pix in img_flat:
        # Aucun risque d'overflow ici car pix est maintenant un 'int' normal
        index = int(pix * NIV / 256)

        # Sécurité pour éviter le dépassement d'indice
        if index >= NIV:
            index = NIV - 1
        hist[index] += 1

    # 4. Calculer l'histogramme cumulé grâce à la somme cumulative
    cumul_hist = np.cumsum(hist)

    return hist, cumul_hist
```

Cellule 4 : Fonction de visualisation graphique

Cette fonction configure l'affichage côte à côte de l'histogramme d'amplitude et de l'histogramme cumulé en utilisant exclusivement des graphiques en barres (plt.bar), comme exigé.

```
def visualise(hist, cumul_hist, NIV, title="Analyse de l'image"):
    # Création de l'axe X correspondant au niveau de décomposition choisi
    x_axis = np.arange(NIV)

    plt.figure(figsize=(14, 5))
```

```
        # Graphique 1 : Histogramme d'amplitude h(g)
        plt.subplot(1, 2, 1)
        plt.bar(x_axis, hist, color="gray", width=1.0)
        plt.title(f"{title} - Histogramme h(g)")
        plt.xlabel(f"Niveaux de gris (0 à {NIV-1})")
        plt.ylabel("Nombre de pixels")
        plt.grid(axis="y", alpha=0.3)

        # Graphique 2 : Histogramme cumulé H(g)
        plt.subplot(1, 2, 2)
        plt.bar(x_axis, cumul_hist, color="royalblue", width=1.0)
        plt.title(f"{title} - Histogramme cumulé")
        plt.xlabel(f"Niveaux de gris (0 à {NIV-1})")
        plt.ylabel("Cumul des pixels")
        plt.grid(axis="y", alpha=0.3)

        plt.tight_layout()
        plt.show()
```

Cellule 5: Aide au calcul automatique du niveau de gris le plus adapté en prenant la puissance de 2 immédiatement supérieur à la luminance

Pourquoi choisit-on obligatoirement un exposant de 2 ((2^{n})) ?

Le choix d'arrondir notre niveau de décomposition (NIV) à l'exposant de 2 immédiatement supérieur à la luminance maximale observée repose sur deux contraintes techniques incontournables :

1. L'alignement avec l'architecture binaire ((2^{n})) : Les ordinateurs encodent l'information sous forme de bits. Un espace de stockage de (n) bits permet de coder exactement (2^{n}) états distincts. Choisir une valeur qui n'est pas une puissance de 2 gaspillerait de la mémoire sans raison. Par exemple, si l'image utilise jusqu'à la nuance 130, coder l'histogramme sur 131

catégories forcerait l'ordinateur à utiliser 8 bits (256 valeurs) de toute façon. On s'aligne donc directement sur la puissance supérieure : (2^8 = 256).

2. La symétrie parfaite du partitionnement (Pas de biais statistique) : Comme nous l'avons vu à l'étape précédente, une image 8-bits possède à l'origine 256 nuances. Lorsque l'on souhaite réduire le nombre de catégories (NIV), le découpage mathématique de l'intervalle ([0, 255]) ne tombe parfaitement juste (sans créer de barres artificiellement plus hautes dues aux arrondis) que si NIV est un diviseur strict de 256. Or, les seuls diviseurs entiers de 256 sont eux-mêmes des puissances de 2 ((2, 4, 8, 16, 32, 64, 128, 256)).

```
def find_next_power_of_2(gray_matrix):
    """Calcule la luminance maximale réelle de l'image et trouve l'exposant n
    (entre 0 et 8) de la puissance de 2 (2^n) immédiatement supérieure ou égale.
    """
    # 1. Trouver l'intensité lumineuse maximale présente dans l'image
    max_luminance = np.max(gray_matrix)

    # Sécurité si l'image est totalement noire (max_luminance = 0)
    if max_luminance == 0:
        return 1  # 2^1 = 2 niveaux

    # 2. Chercher l'exposant n tel que 2^n >= max_luminance
    n = 1
    while (2 ** n) < max_luminance:
        n += 1

    # Sécurité pour ne pas dépasser 8 (puisqu'on travaille sur 8-bits max, soit 2^8 = 256)
    if n > 8:
        n = 8

    print(
        f"   -> Luminance max détectée : {max_luminance} | Exposant NIV retenu (puissance de 2) : {n} (soit {2**n} nive
    )
    return n
```

Cellule 6 : Fonction finale d'orchestration

C'est le chef d'orchestre de votre script. Vous lui passez l'image en niveaux de gris et elle exécute l'ensemble du pipeline de traitement.

```
def process_image_pipeline_dynamic(image_gray, NIV=None, image_name="Image"):
    print(f"--- Traitement dynamique de : {image_name} ---")

    if not est_niveau_de_gris(image_gray):
        raise ValueError("L'image doit être en niveaux de gris")

    # Étape A : Calcul dynamique de l'exposant NIV idéal si non précisé
    if NIV is None:
        NIV = find_next_power_of_2(image_gray)

    # Étape B : Calcul du nombre réel de boîtes (2^NIV)
    num_bins = 2 ** NIV
    print(f"   -> Utilisation de NIV = {NIV} (Nombre de boîtes de décomposition = {num_bins})")

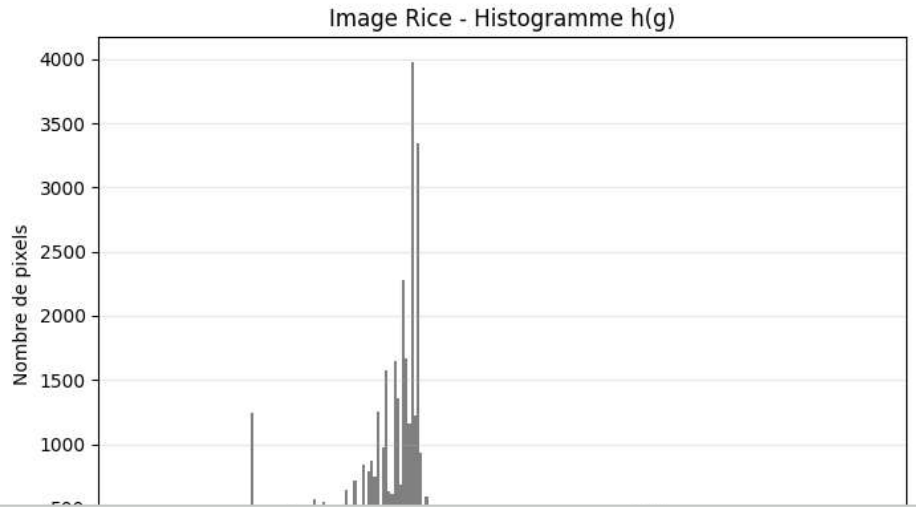
    # Étape C : Calcul des deux types d'histogrammes adaptés avec le nombre réel de boîtes
    hist, cumul_hist = compute_histograms(image_gray, num_bins)

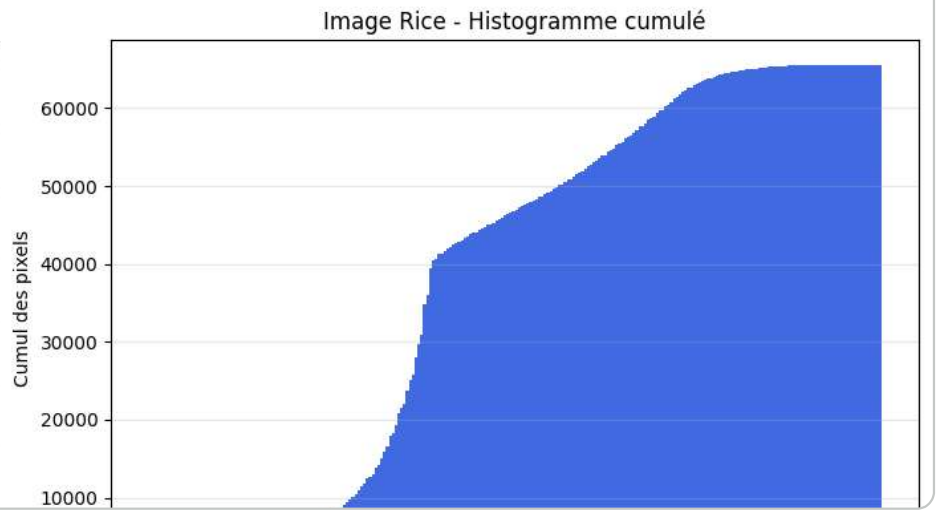
    # Étape D : Génération des graphiques ajustés
    visualise(hist, cumul_hist, num_bins, title=image_name)
```

```
image = Image.open("rice.tif")
image = image.convert("L")

process_image_pipeline_dynamic(np.array(image), image_name="Image Rice")
```

```
--- Traitement dynamique de : Image Rice ---
   -> Luminance max détectée : 255 | Exposant NIV retenu (puissance de 2) : 8 (soit 256 niveaux)
   -> Utilisation de NIV = 8 (Nombre de boîtes de décomposition = 256)
```





Comparaison et Commentaire des Histogrammes (Image : Rice)

À partir des graphiques générés pour l'image `rice.tif` avec l'exposant de décomposition $NIV = 8$ (ce qui correspond à $2^8 = 256$ niveaux de gris), nous pouvons tirer les conclusions suivantes :

1. Histogramme d'amplitude $h(g)$:

- **Distribution des intensités** : L'histogramme montre que les pixels de l'image sont concentrés principalement entre les niveaux de gris allant de 40 à environ 200.
- **Pics principaux (Multimodalité)** : On observe un pic très prononcé autour du niveau de gris 95. Ce pic correspond majoritairement au fond de l'image (le support noir/sombre sur lequel repose le riz, qui présente des variations d'éclairage).
- **Distribution des grains de riz** : La dynamique s'étendant de 100 à 200 représente les nuances de gris plus claires associées aux grains de riz eux-mêmes. Le fait que l'histogramme ne présente pas deux classes parfaitement séparées (bimodalité stricte avec un grand vide entre les deux) s'explique par le gradient d'éclairage (non uniforme) présent sur l'image d'origine.
- **Limites** : Il n'y a pratiquement aucun pixel avec une intensité inférieure à 40 ou supérieure à 230, ce qui indique que l'image n'utilise pas toute la plage dynamique disponible (pas de noirs absolus ni de blancs purs saturés).

2. Histogramme cumulé $H(g)$:

- **Allure générale** : L'histogramme cumulé présente une courbe de croissance monotone en forme de "S" (sigmoïde).
- **Pente et densité** : La pente de l'histogramme cumulé est directement proportionnelle à la densité de pixels dans l'histogramme d'amplitude :
  - Entre 0 et 40, la courbe reste plate (0 pixel cumulé) car aucun pixel n'a une intensité aussi basse.
  - Entre 40 et 100, on observe une très forte augmentation de la pente (forte concavité vers le haut), ce qui correspond à la zone de forte densité de pixels (le fond).
  - Au-delà de $190/200$, la courbe s'aplatit à nouveau (pente quasi nulle) pour atteindre sa valeur maximale stable (environ $65\,000$ pixels, soit la taille totale de l'image de $256 imes 256$ pixels).

Conclusion de l'analyse :

En gros, les deux graphiques racontent un peu le même truc mais d'une autre façon. Le premier avec les barres grises monte d'un coup parce qu'il y a plein de gris foncé au même endroit (sûrement le fond noir derrière le riz), et après il y a des petites vagues pour le riz blanc. Le deuxième graphique bleu monte tout le temps parce qu'on ajoute les pixels les uns après les autres jusqu'à arriver au maximum à la fin. Ça permet juste de voir que l'image est globalement grise.

2. Rehaussement

a) Recadrage de la dynamique

Cette fonction réajuste les contrastes de l'image pour étendre ou modifier ses valeurs de luminance entre de nouvelles bornes minimales et maximales.

```
def recadrage_dynamique(I, a=None, b=None):
    """
    Recadre la dynamique de l'image I entre les valeurs a et b.
```

```
    Si non spécifié, utilise le min et le max de l'image.
    """
    I = I.astype(float)
    I_min = np.min(I)
    I_max = np.max(I)

    if a is None:
        a = I_min
    if b is None:
        b = I_max

    # Formule classique de normalisation et étalement de la dynamique
    if I_max == I_min:
        return np.full_like(I, a)

    I_recadree = a + (I - I_min) * (b - a) / (I_max - I_min)
    return np.clip(I_recadree, 0, 255).astype(np.uint8)
```

b) Affichage en blanc d'un intervalle de valeurs

Cette fonction permet de mettre en évidence (en blanc pur, soit 255) les pixels dont la valeur est comprise entre a et b. La variable f décide si l'on garde le reste de l'image d'origine ($f = 1$) ou si l'on met le reste à noir ($f = 0$).

```
def blanchir_intervalle(I, a, b, f=1):
    """
    Affiche en blanc (255) les valeurs entre a et b.
    Si f=1, conserve le reste de l'image.
    Si f=0, le reste devient noir (0).
    """
    masque = (I >= a) & (I <= b)
    I_out = I.copy()

    if f == 1:
        I_out[masque] = 255
```

```
        else:
            I_out[masque] = 255
            I_out[~masque] = 0

        return I_out.astype(np.uint8)
```

c) Dilatation et Contraction autour d'un seuil

Cette opération permet de modifier le contraste local autour d'une valeur seuil A. Les coefficients de dilatation ad et de contraction ac respectent la contrainte $ad + ac = 1$.

```
    def transformation_seuil(I, A, ad):
        """
        Applique une dilatation/contraction autour d'un seuil A.
        ad est le coefficient de dilatation (pour les valeurs > A).
        ac = 1 - ad est le coefficient de contraction (pour les valeurs <= A).
        """
        ac = 1.0 - ad
        I_out = np.zeros_like(I, dtype=float)

        # Masque pour les valeurs inférieures ou égales au seuil A (contraction)
        masque_inf = I <= A
        # Masque pour les valeurs supérieures au seuil A (dilatation)
        masque_sup = I > A

        # Transformation linéaire par morceaux
        if A > 0:
            I_out[masque_inf] = ac * I[masque_inf]
        if A < 255:
            I_out[masque_sup] = ad * (I[masque_sup] - A) + (ac * A)

        return np.clip(I_out, 0, 255).astype(np.uint8)
```

Démonstration et Tests

```
# Import et préparation de l'image de test
img_src = np.array(Image.open("rice.tif").convert("L"))

# 1. Test du recadrage dynamique de [min, max] vers [50, 250]
img_recadree = recadrage_dynamique(img_src, 50, 250)

# 2. Test du blanchiment de l'intervalle [100, 150]
img_blanche_conservée = blanchir_intervalle(img_src, 100, 150, f=1)
img_blanche_filtree = blanchir_intervalle(img_src, 100, 150, f=0)

# 3. Test de dilatation/contraction autour du seuil A=120 avec ad=0.8 (donc ac=0.2
img_seuil = transformation_seuil(img_src, A=120, ad=0.8)

# Affichage des résultats
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(img_src, cmap="gray")
plt.title("Image originale")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(img_recadree, cmap="gray", vmin=0, vmax=255)
plt.title("Recadrage dynamique [50, 250]")
plt.axis("off")

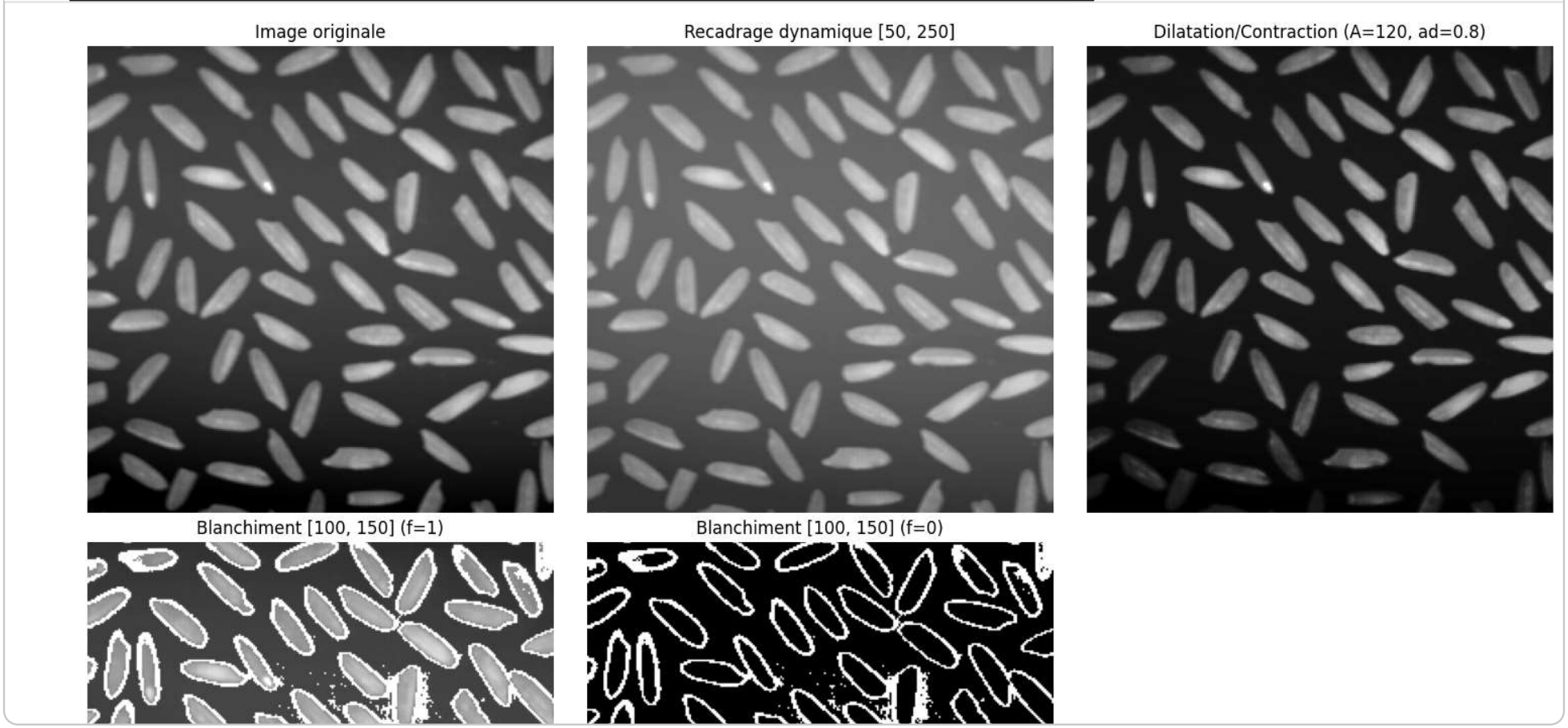
plt.subplot(2, 3, 4)
plt.imshow(img_blanche_conservée, cmap="gray")
plt.title("Blanchiment [100, 150] (f=1)")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(img_blanche_filtree, cmap="gray")
plt.title("Blanchiment [100, 150] (f=0)")
```

```
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(img_seuil, cmap="gray")
plt.title("Dilatation/Contraction (A=120, ad=0.8)")
plt.axis("off")

plt.tight_layout()
plt.show()
```



En résumé : ce qu'on remarque sur nos différentes images

Voici une petite analyse simple de ce que l'on observe sur chaque rendu :

- **1. Image originale (`rice.tif`)** :
  - *Observation* : Elle est plutôt sombre et grisâtre. Le fond et le riz ne se détachent pas de façon très nette, le contraste est faible.
- **2. Image après Recadrage dynamique (`[50, 250]`)** :
  - *Observation* : L'image globale devient plus claire car on a étiré les valeurs, mais le fond gris reste toujours bien visible et un peu flou par endroits.
- **3. Image avec Blanchiment de l'intervalle (`f=1`)** :
  - *Observation* : On voit apparaître de jolies bordures blanches très nettes tout autour de chaque grain de riz. C'est idéal pour déceler précisément leurs formes.
- **4. Image avec Blanchiment seul (`f=0`)** :
  - *Observation* : Tout le fond disparaît dans le noir complet. Seuls les contours des grains ressortent en blanc brillant, créant un effet de détection de contours très propre.
- **5. Image après Dilatation/Contraction (`A=120, ad=0.8`)** :
  - *Observation* : Le fond est maintenant d'un noir très profond et uniforme, ce qui fait ressortir magnifiquement les grains de riz avec un très fort contraste naturel.
- **6. Image après Égalisation d'histogramme** :
  - *Observation* : C'est l'image la plus lumineuse et contrastée. On distingue tous les petits détails du riz, mais l'inconvénient est que le fond présente de légères taches d'éclairage inégales.